Digital Cousin: A Single-Asset, Edge-First Digital Twin with a Natural-Language Copilot for Legacy MSME Machine Monitoring

1st Samarth D Kolur
Dept. of Computer Science and Technology
Dayananda Sagar University
Bengaluru, India
samarthdkolur1@gmail.com

2nd Manoj C
Dept. of Computer Science and Technology
Dayananda Sagar University
Bengaluru, India
manojchandrashekhar.058@gmail.com

3rd Yashaswini R
Dept. of Computer Science and Technology
Dayananda Sagar University
Bengaluru, India
rygowda1703@gmail.com

4th Dr. Ramandeep Kaur
Dept. of Computer Science and Technology
Dayananda Sagar University
Bengaluru, India
ramandeep-ct@dsu.edu.in

5th Dr. Danthuluri Sudha
Dept. of Computer Science and Technology
Dayananda Sagar University
Bengaluru, India
danthulurisudha15@gmail.com

***Abstract*—Micro, Small and Medium Enterprises (MSMEs) contribute 30.1% of India's gross domestic product and 45.73% of its exports, yet most of their production assets are legacy machines with no industrial data interface. Commercial digital twin platforms cannot reach these assets, because they presuppose protocol-compliant machinery, continuous cloud connectivity and dedicated IT staff. This paper presents Digital Cousin, a digital twin designed for that setting and formalised as a three-property paradigm: single-asset in scope, edge-first in execution and natural-language accessible in interface. The system retrofits one high-risk rotary machine with a non-invasive MEMS accelerometer and a 1-Wire temperature probe, synchronises a structured state object on a Raspberry Pi 4, and derives anomaly score, health index, confidence flag and a remaining-useful-life fraction on-device using an Isolation Forest baseline, a 1D convolutional autoencoder and a four-feature ridge regressor exported to ONNX. Machine health is exposed to a floor operator through a quantized small language model served locally, whose answers are constrained to retrieved state objects and which refuses to answer when its retrieval context is empty, stale or flagged out-of-distribution. The complete deployment costs approximately INR 20,000 in one-time hardware, with no software licence and no cloud dependency. We report the system design, the retrofit protocol and the evaluation methodology; quantitative results from the pilot deployment are reported against the measurement plan given here.**

***Index Terms*—digital twin, MSME, brownfield retrofit, edge computing, TinyML, predictive maintenance, anomaly detection, small language models, retrieval-augmented generation, Industry 4.0**

I. Introduction

India's Micro, Small and Medium Enterprise (MSME) sector is the structural base of the country's manufacturing output. Government figures place the sector's gross value added at 30.1% of national GDP, 35.4% of manufacturing output and 45.73% of exports [36]. The same sector is also the least instrumented part of that economy. Its capital stock is dominated by lathes, compressors, pumps and induction motors that predate any notion of an industrial data interface, and the financial headroom to replace them does not exist. Assessments of MSME finance in India put total credit demand at roughly INR 69.3 lakh crore against a formal supply near INR 14.5 lakh crore, leaving an addressable gap on the order of INR 25.8 lakh crore [37].

Digital twin technology has matured substantially over the same period [1], and reference architectures now exist for twin-based predictive maintenance [3] built on the five-level cyber-physical hierarchy that underpins most Industry 4.0 deployments [2]. These architectures are sound, and they are also inapplicable to this population of assets. Each assumes a stream of structured telemetry arriving over OPC-UA, MTConnect, Modbus or a vendor-native stack, and a legacy lathe emits none of these. Even work that explicitly targets cost reduction in twin construction, such as open-architecture reconfiguration via RESTful middleware [4], validates on actuators that are already protocol-compliant. The barrier is therefore not the price of the twin but the absence of anything for the twin to read.

Three further constraints compound the first. Cross-country studies of Industry 4.0 adoption identify cost, deficient digital infrastructure and skill shortage as the dominant barriers in manufacturing, and locate the binding constraint not in awareness but in the absence of an affordable implementation path [33]. Bibliometric review of three decades of SME digitalisation research confirms that firms in developing economies remain disproportionately stalled at the awareness stage rather than the implementation stage [34]. The interface itself also

excludes the intended user, since a dashboard of kurtosis trends and reconstruction-error plots presumes a reader who can interpret them, which the shop floor of a ten-person workshop generally does not contain.

Recent progress in small language models (SLMs) suggests a way past that last barrier. Models in the 0.5B to 2B parameter range now run usefully on commodity ARM hardware [21], [22], systematic measurement of some seventy open SLMs has established their on-device latency and memory envelopes [39], and frameworks for training and deploying them at the edge are available [23]. Pairing such a model with retrieval over live machine state offers a plausible operator interface that requires no charts and no training. The industrial literature has begun to exploit this. ChatCNC grounds a conversational monitor in a live IIoT database and reports 93.3% accuracy on complex production-tracking queries [25]. IoT-LLM reports a 49.4% average improvement on sensory reasoning tasks by combining retrieval augmentation with chain-of-thought prompting [24]. BearLLM unifies nine bearing diagnosis benchmarks under a single multimodal model [26], and an MTConnect-based twin has been equipped with a multi-agent natural-language interface for legacy and smart machines alike [6].

Every one of these systems, however, places the language model in the cloud or on server-class hardware. For a firm with intermittent connectivity and no appetite for a metered per-query bill, that placement is itself the obstacle. What is missing is a twin that is narrow enough, cheap enough and self-contained enough to sit on one machine in one workshop and still answer a question in plain language.

This paper presents Digital Cousin, built for that intersection of constraints, and makes the following contributions.

1) **A formal three-property paradigm.** We define the Digital Cousin as a twin that is single-asset in scope, edge-first in execution and natural-language accessible in interface, and argue in Section III that this is a distinct design point rather than a degraded enterprise twin.
2) **A complete four-layer architecture and retrofit protocol** for a non-instrumented rotary asset, with a published bill of materials totalling approximately INR 20,000 per deployment and no recurring cost (Section IV).
3) **An edge analytics pipeline** combining statistical feature extraction, an Isolation Forest baseline, a 1D convolutional autoencoder and a deliberately minimal ridge regressor for remaining useful life, all exported to a single ONNX runtime and tied together by an explicit model manifest (Section V).
4) **A retrieval-grounded copilot** whose context-validity gate routes empty, stale or out-of-distribution retrievals to a deterministic rule-based response rather than to the language model (Section VI).
5) **A full evaluation methodology and comparison frame** (Sections VII and VIII) stating every metric, target and baseline in advance of measurement.

II. Related Work

A. Digital Twin Architectures for Constrained Settings

The modern digital twin literature is organised around the integration of physical and cyber spaces [1] and, in manufacturing, around the 5C hierarchy of connection, conversion, cyber, cognition and configuration [2]. Van Dinter et al. synthesised 42 twin-based predictive maintenance studies into a reference architecture evaluated across three case studies [3], giving the field a rigorous blueprint whose applicability nonetheless begins at the point where structured telemetry already exists. Leng et al. showed that open-architecture machine tools and REST-based IIoT middleware can reconfigure a twin rapidly and cheaply [4], which establishes that twin cost is compressible, although the demonstration rests on tools designed to be read.

The closest published approach to the present setting is that of Krommes and Tomaschko, who propose a conceptual twin framework for brownfield, small-volume SME production explicitly scoped around low investment, minimal IT prerequisites and user-oriented skill requirements, grounded in value and material flow analysis [5]. Their framework reaches monitoring, but it does not extend to predictive analytics or to an operator-facing language interface. Gautam et al. come at the problem from the opposite direction, standardising legacy machine data through MTConnect and overlaying a four-agent LLM interface on a high-fidelity twin [6]. That result addresses legacy assets and natural language together, but requires an MTConnect-capable server, a cloud-hosted model and a 3D visualisation environment.

B. Bearing Diagnostics and the Datasets Used Here

Rolling-element bearing diagnosis is the best-characterised failure mode in rotary machinery. The signal-processing basis for it, covering the pseudo-cyclostationary nature of bearing signals, spectral kurtosis band selection and envelope analysis, is set out in the tutorial treatment of Randall and Antoni [9]. Two datasets dominate empirical work. The Case Western Reserve University (CWRU) bearing dataset provides seeded inner-race, outer-race and ball faults at 12 and 48 kHz. Smith and Randall's benchmark study is essential reading here, because it establishes that CWRU records differ in diagnosability and that some are not reliably diagnosable at all [7], which bears directly on how the set should be partitioned. The IMS run-to-failure dataset, introduced alongside wavelet-filter-based weak signature detection by Qiu et al. [8], provides continuous degradation histories at 20 kHz and is the natural source for remaining-useful-life work.

Deep architectures on these datasets now report very high accuracy. The survey of deep learning for machine health monitoring by Zhao et al. catalogues the model families and, importantly for our purposes, identifies a class of lightweight 1D convolutional and autoencoder designs that retain competitive accuracy at sharply reduced cost [10]. Hu et al. apply a masked 1D convolutional autoencoder to bearing diagnosis inside a twin-enabled IIoT setting [11]. Yuan et al.

use an autoencoder as a denoising front-end to a CNN and report 99.83% accuracy on CWRU under noise-free conditions [13], and Fu et al. extract spatial and temporal structure in parallel with a CNN and LSTM network evaluated across two datasets [14]. These results are strong, and they are obtained at sampling rates of 12 kHz and above on server-class compute, neither of which a sub-INR-300 MEMS accelerometer on a Raspberry Pi can supply. The Isolation Forest [12], by contrast, is cheap enough to train on a microcontroller, which is why it recurs throughout the edge literature and why we adopt it as our operational baseline.

C. Edge Inference and TinyML

TinyML has made on-device inference routine on microcontroller-class hardware, with compression, distillation and architecture search bringing capable models under a megabyte [18]. Njor et al. review the full TinyML stack for predictive maintenance and find the field's attention unevenly distributed, with models and optimisation well served while data optimisation and representative datasets are not [15]. Reviews of edge computing for machine signal processing identify the recurring obstacles as resource limits, communication overhead and the absence of standard deployment toolchains [19], and a systematic review of TinyML in industrial predictive maintenance concludes that the technology is a plausible bridge for resource-constrained settings but lacks a demonstrated end-to-end deployment in an MSME manufacturing context [35].

Two systems define the achievable envelope. Antonini et al. run an unsupervised Isolation Forest on an ESP32 for submersible pump monitoring, training on-device in 1.2 to 6.4 s and inferring in under 16 ms with 50 trees in 80 KB of RAM [16], and, critically for a context with no fault history, learning normality without labelled faults. Chen et al. go further with LOPdM, harvesting vibration energy through a self-powered sensor so that the node needs neither mains power nor a network [17]. Pournazari et al. demonstrate with LiRUL that physics-constrained, feature-compressed remaining-useful-life prediction fits an edge budget, reporting 0.15M parameters, under 5 MB of memory, sub-second inference and up to 21% RMSE reduction from domain features [20]. Their validation is on battery cells rather than rotating machinery, but the resource envelope transfers. What none of these systems provides is an operator interface. Each ends at an anomaly flag or a scalar, on the implicit assumption that a trained technician will read it.

D. Language Interfaces for Industrial Data

Industrial LLM work has moved quickly from expert tooling toward operator interaction. Xia et al. use a four-role multi-agent system to parametrise process simulations inside a twin from natural-language intent, and caution explicitly that current models lack the deterministic physical fidelity and closed-loop stability that autonomous intervention would require [27]. We take that finding as a design constraint and honour it by making our copilot strictly read-only. Gill et al. pair LLM agents with a twin for fault handling in process plants, using the twin both as a knowledge repository for prompting and as a validation sandbox for proposed corrective actions [28]. On the grounding side, IoT-LLM establishes that retrieval augmentation plus chain-of-thought substantially improves sensory reasoning [24], ChatCNC demonstrates that live retrieval from an IIoT database yields high query accuracy in production monitoring [25], and BearLLM shows that a unified vibration representation can carry one set of weights across nine diagnosis benchmarks [26].

The common structure of these systems is a cloud-hosted or server-class model, a curated document corpus or a fine-tuning dataset, and an evaluation reported as task accuracy. The broader question of whether a generated answer is faithful to its retrieval context has been studied carefully, but over prose rather than telemetry. FaithBench assembles challenging summarisation hallucinations from ten models across eight families with span-level annotation [29], RAGTruth provides nearly 18,000 word-level annotated RAG instances across question answering, summarisation and data-to-text [31], and RAGAs automates reference-free scoring of faithfulness, answer relevance and context relevance [30]. The hallucination survey of Huang et al. organises this space by separating factuality from faithfulness [32]. Extending such evaluation to structured numeric telemetry is a problem we scope to future work in Section XI rather than address here.

E. Positioning

Table I scores representative prior work against the five properties this deployment requires, drawn from the constraints enumerated in Section I. Individual properties are well served and several works satisfy three. We are not aware of prior work that satisfies all five.

III. The Digital Cousin Paradigm

We use the word cousin rather than twin deliberately. A twin connotes fidelity, meaning a virtual replica intended to mirror its physical counterpart closely enough to substitute for it in simulation. Nothing in this system attempts that. What it offers instead is a partial, cheap, deliberately narrow relative of the asset, with enough shared structure to answer one question usefully and no pretence of completeness. The paradigm is defined by three properties.

P1, single-asset scope. The unit of deployment is exactly one machine, chosen because it is the bottleneck or the highest-consequence failure point. This is a positive design choice, not a budget-driven truncation. Plant-wide architectures spend their complexity on topology, asset registries, cross-asset correlation and multi-tenant data models, all of which is overhead for a firm whose production depends on one lathe. Narrowing scope to a single asset collapses the state model to one object, removes the need for an asset ontology of the kind that makes knowledge-graph-driven twins expensive to maintain, and brings the compute budget inside a single-board computer.

TABLE I
REPRESENTATIVE PRIOR WORK AGAINST THE FIVE REQUIRED PROPERTIES

Work	P1	P2	P3	P4	P5
Tao et al. [1]	N	N	N	N	N
Lee et al. [2]	N	N	N	N	N
van Dinter et al. [3]	N	N	N	N	N
Leng et al. [4]	N	N	N	N	P
Krommes & Tomaschko [5]	N	N	P	N	P
Gautam et al. [6]	N	N	P	Y	N
Zhao et al. [10]	Y	P	N	N	N
Hu et al. [11]	Y	N	N	N	N
Yuan et al. [13]	Y	N	N	N	N
Fu et al. [14]	Y	N	N	N	N
Njor et al. [15]	P	P	N	N	Y
Antonini et al. [16]	Y	Y	N	N	Y
Chen et al. [17]	Y	Y	N	N	Y
Lu et al. [19]	N	P	N	N	N
Pournazari et al. [20]	Y	Y	N	N	Y
Kandala et al. [23]	N	Y	N	P	Y
An et al. [24]	Y	N	N	Y	N
Jeon et al. [25]	Y	N	N	Y	N
Peng et al. [26]	Y	N	N	Y	N
Xia et al. [27]	N	N	N	Y	N
Gill et al. [28]	N	N	N	Y	N
Raj et al. [33]	N	N	Y	N	N
Ooko & Karume [35]	P	Y	P	N	Y
Digital Cousin	**Y**	**Y**	**Y**	**Y**	**Y**

Y denotes a demonstrated property, P a partially addressed one or one addressed with a structural dependency that limits transfer, and N a property not addressed. **P1** single-asset scope; **P2** edge-only execution; **P3** MSME deployment context; **P4** natural-language operator interface; **P5** low-cost hardware compatibility.

P2, edge-first execution. All functions required for the system to be useful, namely acquisition, feature extraction, inference, storage, alerting, dashboard and the language interface, execute on the edge node with no network egress. Cloud connectivity is permitted only as an optional accelerator for the copilot and never as a dependency. This is what distinguishes the design from edge-assisted architectures that localise inference but retain the cloud for retraining or visualisation. An intermittent connection in an industrial area degrades such systems to unavailability, and metered egress converts a one-time purchase into a subscription.

P3, natural-language accessibility. The primary interface for the operator is a question in their own words, answered in plain language. The dashboard is retained for the owner and the maintenance technician, but it is not the path by which a machine operator learns that their machine is in trouble. The constraint this places on the implementation is severe. An interface that cannot be cross-checked by its user against a chart must not state numbers it did not retrieve, which is why the context-validity gate of Section VI is treated as a correctness requirement rather than a convenience.

A. State Object

The twin is realised concretely as a single structured state object, re-synchronised from sensor readings at a configurable interval, whose fields are given in Table II. The object is the system's sole source of truth. The edge layer is its only writer, and the dashboard, the alert module and the copilot are readers. Fields derived from models are nullable and remain null until trained artefacts are present, so that the schema never carries a placeholder that a consumer could mistake for a measurement.

TABLE II
DIGITAL TWIN STATE OBJECT

Field	Type	Source and meaning
`asset_id`	string	Monitored machine identifier
`timestamp`	ISO-8601	Acquisition time, UTC
`vibration.rms_g`	float	RMS of AC-coupled signal
`vibration.kurtosis`	float	Fourth standardised moment
`vibration.crest_factor`	float	Peak over RMS
`vibration.peak_to_peak_g`	float	Window peak-to-peak
`vibration.sampling_hz`	int	Achieved burst sample rate
`temperature_c`	float	1-Wire probe, °C
`anomaly_score`	float?	Fused model output, 0 to 1
`health_index`	float?	Calibrated scalar, 0 to 1
`remaining_useful_life`	float?	Life-fraction proxy, 0 to 1
`model_confidence`	enum?	`high` or `low`
`alert_level`	enum?	`normal`, `warning` or `critical`

Types marked with a question mark are nullable and stay null until the corresponding trained artefact is loaded. The `remaining_useful_life` field is a life-fraction proxy, not a wall-clock estimate; see Section V.

IV. SYSTEM ARCHITECTURE

Fig. 1 shows the four layers. We describe each in turn, emphasising the choices that the MSME constraint forced.

A. Physical Layer and Retrofit

Vibration is sensed with an ADXL345 three-axis MEMS accelerometer on a GY-291 type breakout, driven over SPI with the bus held at 3.3 V so that all logic levels match. Temperature is sensed with a waterproof DS18B20 probe on a 1-Wire bus. The probe shell contains no pull-up, so the data line requires one externally. A 4.7 kΩ resistor is the correct permanent solution; the host's internal GPIO pull-up will also enumerate the device, but it is configured at runtime and is lost on reboot, so it is adequate for bench bring-up and not for an installed system. Both sensors mount non-invasively, using a magnetic or clamp fixture on the bearing housing and a probe strapped to the motor casing, so that no hole is drilled and no warranty or alignment is disturbed. Rigid mounting matters more than sensor grade, because a compliant mount low-pass-filters the very transients the system is looking for.

Two assembly details earn a place in the protocol because they cost us bring-up time directly. First, the accelerometer breakout's header pins must be soldered rather than press-fitted. A header resting in its holes will make and break every connection on the module, power included, which presents as an intermittent fault rather than a dead one, and a module destined to sit on a vibrating machine will not tolerate it. Second, the device-identity read described below is the test that localises this class of fault, because a module with an unreliable bus connection returns an all-zero identity while the rails measure correct.

The accelerometer's register path is implemented against the datasheet with a 3.9 mg/LSB scale factor and an explicit `POWER_CTL`, `DATA_FORMAT` and `BW_RATE` initialisation sequence, and a device-identity read is used as the installation

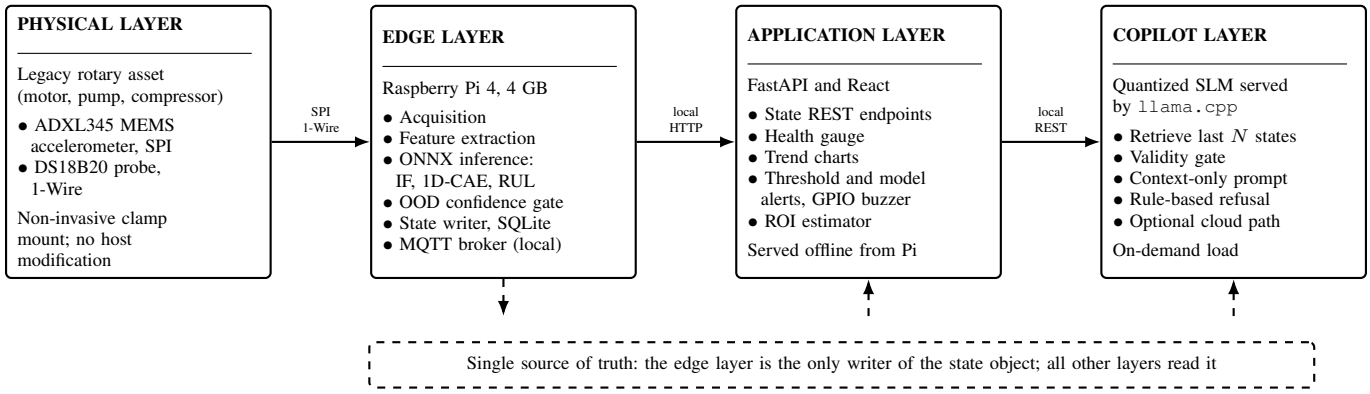


Fig. 1. Four-layer Digital Cousin architecture. Every element shown executes on the single edge node or on the sensors attached to it. The optional cloud path in the copilot layer is the only route that ever leaves the device, and the system is fully functional without it.

smoke test. We report the achieved rate honestly, because it differs from the part's ceiling. The ADXL345 supports output data rates to 3,200 Hz in SPI mode, whereas the current acquisition path takes a 64-sample burst per read cycle at approximately 800 Hz, which is what a synchronous polling loop inside an asynchronous service process sustains without real-time buffering. Sustaining the hardware ceiling is a real-time scheduling problem that we treat as empirical tuning work against the pilot machine rather than as a solved claim. The consequence is bounded and stated in Section X. High-frequency bearing signatures above roughly 5 kHz, which envelope analysis of the kind described in [9] would target, are out of reach, and the system is scoped to low-frequency and mid-frequency mechanical degradation detectable through amplitude and shape statistics.

A battery-backed DS3231 real-time clock completes the physical layer. The Raspberry Pi has no on-board clock, so without one it restores a stale time at boot and depends on reaching a network time source to correct it. That dependency is incompatible with P2, and it would undermine the state object directly, since every record is keyed by the `timestamp` field of Table II and an alert history with an invented time is not evidence. The clock shares the I2C bus, draws negligible current and adds roughly INR 200 to Table IV.

B. Edge Layer

A Raspberry Pi 4 with 4 GB of RAM hosts acquisition, feature extraction, inference, persistence and messaging. Features are computed per burst: RMS of the AC-coupled signal, kurtosis, crest factor and peak-to-peak. AC coupling matters, because the window mean is dominated by the roughly 1 g gravity component at any real mount orientation, and subtracting it before computing RMS is the difference between measuring vibration and measuring mount angle.

Persistence uses SQLite in two deliberately separate tables. `raw_readings` records what the sensor actually reported, while `state_history` records the full state object including model-derived fields. Keeping them apart means the raw record never acquires placeholder values for quantities that did not exist when it was written. Two MQTT topics mirror the split on a local broker, so that no downstream consumer can mistake an unenriched reading for a complete state object.

TABLE III
EDGE NODE MEMORY BUDGET (RASPBERRY PI 4, 4 GB)

Process	Allocation
Raspberry Pi OS Lite (64-bit)	400 MB
Acquisition service and MQTT broker	150 MB
ONNX Runtime and loaded models	200 MB
SQLite and write buffer	100 MB
API and dashboard service	100 MB
Quantized SLM (on-demand, Q4_K_M)	700 MB
Peak total	**1,650 MB**
Headroom	**2,350 MB**

C. Application and Copilot Layers

Three FastAPI services and a React single-page application compose the user-facing system, all served from the Pi over the local network. The dashboard renders a health gauge, vibration and temperature trends, an active-alert panel, a filterable history log and a remaining-useful-life readout, using inline SVG rather than a charting dependency to keep the offline bundle small. Alerting combines configurable thresholds with the model's own alert level, surfacing both rather than letting either mask the other, and drives a GPIO buzzer for the floor. An ROI estimator converts operator-supplied downtime cost and annual unplanned hours into a projected saving, entirely client-side.

The copilot is described in Section VI. Architecturally its one notable property is that it is not resident. It loads the quantized model on a query and unloads afterwards, which trades a few seconds of latency for the guarantee that it never contends for memory with the inference pipeline. Table III gives the resulting steady-state budget.

D. Bill of Materials

Table IV gives the full per-deployment bill of materials for the shop-floor configuration, totalling approximately INR 20,000 including contingency. Two points about this figure. First, it is one-time. There is no licence, no subscription and no metered egress, so the five-year cost of ownership is the

TABLE IV
PER-DEPLOYMENT BILL OF MATERIALS, SHOP-FLOOR CONFIGURATION

Component	Part or specification	INR
Vibration sensor	ADXL345 breakout, SPI	250
Temperature sensor	DS18B20 waterproof	200
Pull-up and passives	4.7 kΩ, decoupling	50
Edge compute	Raspberry Pi 4, 4 GB	6,500
Real-time clock	DS3231, battery-backed	200
Power supply	5 V, 3 A USB-C	800
Power buffering	UPS HAT, 2 cells	1,600
Storage	32 GB Class-10 card	400
Operator HMI	7-inch touchscreen	4,800
Enclosure	IP54 box, DIN rail	1,200
Sensor mounting	Magnetic fixture	600
Interconnect	3 m shielded cable	500
Audible alert	Piezo buzzer, driver	150
Thermal	Heatsink case, fan	600
Consumables	Jumpers, fasteners	350
Subtotal		**18,200**
Contingency and spares	10%	1,820
Total per deployment		**20,020**

Indicative Indian retail prices at the project's procurement window, not vendor quotations. Software cost is zero, since the entire stack is open source. The headless configuration is approximately INR 12,200, and the sensing and compute core alone is approximately INR 7,600.

purchase price plus electricity. Second, it is the *full* configuration. The sensing and compute core alone is roughly INR 7,600, and a headless variant without the operator touchscreen, power buffering and active cooling comes to approximately INR 12,200. We quote the INR 20,000 figure throughout because the touchscreen is what makes P3 real on a factory floor where the operator has no laptop, and excluding it would understate what deployment actually costs.

V. EDGE ANALYTICS PIPELINE

The pipeline is trained off-device on a development machine and deployed as ONNX artefacts, with a JSON manifest acting as the contract between the training service and the inference service. The manifest carries model paths, window length, feature order, out-of-distribution bounds and recorded evaluation scores, which means the inference side never hard-codes an assumption about how the model was built.

A. Datasets and Their Distinct Roles

CWRU [7] and IMS [8] are used for two different purposes and are deliberately never merged into one training set. They sample at 12 kHz and 20 kHz respectively, their fault semantics differ, and the reliability of their labels differs. Treating them as interchangeable would hide all three differences.

CWRU trains the anomaly detectors. We use 64 real files, comprising the normal baseline set plus the 12 kHz drive-end fault set, reading each record's drive-end channel. Because the set contains far more fault files than normal files, a naive window extraction yields a training set that is roughly 94% faulty. We therefore cap windows per file with a proportionally higher allowance for normal files, which brings the class balance back to something a normality model can learn from, yielding 35,311 windows at 17,311 normal against 18,000 faulty. Smith and Randall's benchmark [7] informs this handling directly, since their finding that CWRU records vary in diagnosability is the reason file selection and partitioning are treated as methodological choices rather than bookkeeping.

Fig. 2 shows the resulting class-conditional distribution of each of the four features, with the out-of-distribution band of Section V overlaid. The two amplitude statistics separate the classes most strongly and the two dimensionless shape statistics overlap substantially, which is the empirical reason the detectors consume all four rather than any single one. The band tracks the normal class in every panel, which is the behaviour the gate is designed to have.

IMS trains the remaining-useful-life regressor. Its run-to-failure structure carries no per-snapshot fault label, so each sampled file's position in its run's chronological sequence becomes a continuous life-fraction-remaining target descending from 1.0 to 0.0. The split is by *file*, not by window, because windows drawn from the same file are near-duplicates and splitting on them would leak the test set into training.

B. Models

Isolation Forest. An Isolation Forest [12] over the four statistical features, trained on CWRU normal-class windows, is the operational baseline. It is chosen for the same reason it recurs in the edge literature [16]: it is small, it trains unsupervised, and it needs no fault history, which matters because an MSME has no labelled fault records for its own machine. Implementation is via scikit-learn [38].

1D convolutional autoencoder. A 1D convolutional autoencoder over raw vibration windows provides the primary detector, following the lightweight-architecture family identified in [10] and applied to bearing diagnosis in [11]. Reconstruction error is thresholded against normal-class windows. The window length defaults to 64 samples to match the acquisition burst size exactly, so the deployed model consumes the acquisition output without an intervening buffering redesign, and a final interpolate-to-input-length stage makes the network tolerant of other window lengths. We note the cross-rate assumption plainly. Training windows match the deployment window in *sample count*, not in physical duration, so a CWRU window and a field window span different timescales. The four features the detectors consume are amplitude and shape statistics rather than timescale-dependent quantities, which is why we consider the assumption defensible, but it is an assumption, and validating it against the pilot machine is part of Section VII.

Remaining-useful-life regression. This uses a ridge regressor over the same four features the Isolation Forest already computes. It is a deliberate choice of the smallest adequate model rather than the strongest available one. The exported artefact is 252 bytes, and inference costs one additional runtime call on a four-element vector, so the prognostic channel adds effectively nothing to the edge budget. LiRUL [20] demonstrates the general principle, that feature compression plus domain constraint fits an edge envelope, at a scale two orders of magnitude larger than ours. We report the output

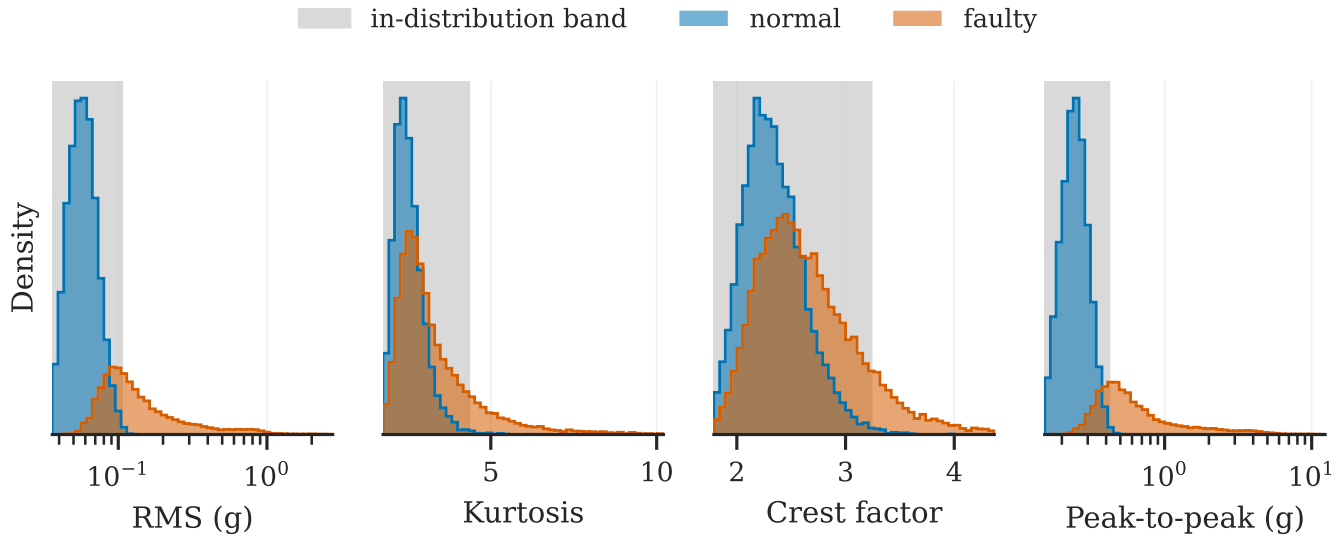


Fig. 2. Class-conditional distributions of the four statistical features over 35,311 CWRU windows (17,311 normal, 18,000 faulty). The shaded band is the manifest's recorded in-distribution interval, spanning the 0.5th to 99.5th percentile of the training distribution, which the out-of-distribution gate of Section V enforces at inference time. RMS and peak-to-peak use a logarithmic axis, since the two classes differ by orders of magnitude on those features; kurtosis and crest factor are dimensionless and remain linear. Densities are normalised per class, so the vertical scale reflects shape rather than class size.

as a life-fraction proxy and not as a time-to-failure estimate, because that is what training against chronological position in a run actually yields. One implementation detail is worth stating, because it affects reproduction. The exported graph is the bare linear model and carries no clamp, so a reader scoring the artefact in isolation will see predictions outside the unit interval on out-of-distribution inputs. Both runtime consumers clamp to $[0, 1]$, which is the behaviour Table II specifies and the behaviour the figures reported here reflect.

Health index. A weighted linear combination of reconstruction error and Isolation Forest score, default 50/50, normalised to $[0, 1]$ and calibrated against a baseline recording so that approximately 1.0 is nominal and 0.6 is the alert boundary, with a sensitivity parameter expressed in standard deviations above baseline at full anomaly (default four). Calibration is sourced from whichever dataset trained the detectors. IMS windows are not cross-fed through CWRU-trained models for calibration, as that would stack a second unvalidated rate assumption on the first.

Out-of-distribution gate. Before any prediction is published, each input feature is checked against the training distribution's 0.5th and 99.5th percentile band. The interval is two-tailed by design, because an anomalously *low* reading, such as a detached sensor or a stopped machine misread as healthy, is as informative as a high one. A feature outside the band sets the model confidence field to low. Crucially the prediction is flagged, not suppressed. An operator shown an uncertain reading is better served than one shown a confident false normal, and the same flag is what causes the copilot to refuse rather than narrate.

VI. Retrieval-Grounded Copilot

The copilot answers operational questions, such as whether the machine is healthy, what changed, and why it alerted, in plain language over a quantized SLM served by `llama.cpp` on the Pi. TinyLlama-1.1B at Q4_K_M quantization [21] is the reference configuration at roughly 700 MB resident, and Qwen2.5-0.5B [22] is the lower-footprint alternative. The choice of this size class is informed by systematic on-device measurement of SLM latency and memory [39] and by edge deployment frameworks [23]. Expected throughput on an ARM Cortex-A72 is 1 to 3 tokens per second, giving 5 to 15 s for a typical answer, which is acceptable for diagnostic question answering and explicitly not acceptable for control. Consistent with the caution against autonomous LLM intervention in [27], we keep the copilot strictly read-only.

Fig. 3 gives the query path. Four aspects of it are load bearing.

Retrieval is structured, not textual. The last N state objects, ten by default, are fetched from the local API and serialised into the prompt as structured context. There is no document corpus, no embedding model and no vector index, because the retrieval context is the telemetry itself. This distinguishes the setting from IoT-LLM's curated domain-knowledge retrieval [24] and from BearLLM's fixed paired corpus [26].

Retrieval failure is a first-class outcome. The retrieval client returns an empty list rather than raising on any failure, because an empty retrieval is exactly the condition that should trigger refusal. A transport error must not become an exception that some outer handler converts into a generic apology. It

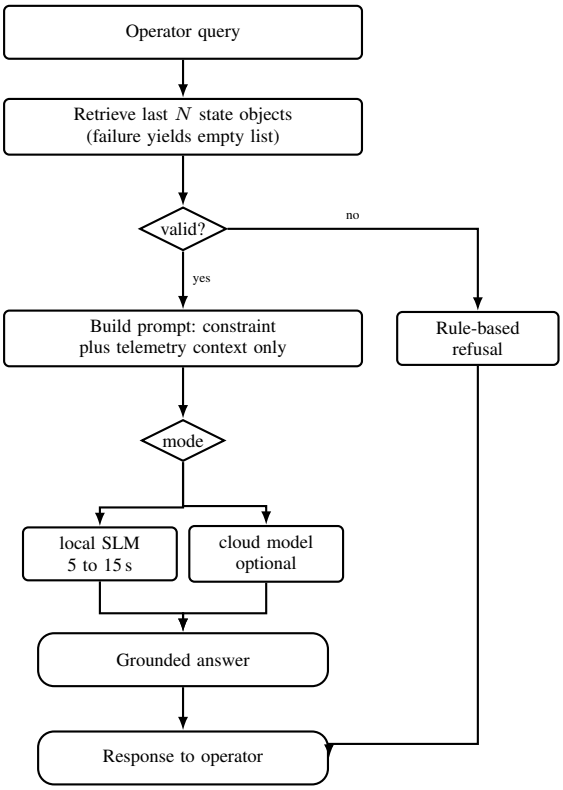


Fig. 3. Copilot query path. The validity gate runs before the language model, so an empty, stale or out-of-distribution context yields a deterministic refusal rather than a generated answer.

must become a refusal with a stated reason.

The validity gate precedes the model. Context is checked for emptiness, for staleness beyond a five-minute horizon, and for an out-of-distribution confidence flag. Any of these routes the query to a deterministic rule-based response, and the language model is never invoked. The target is categorical: no model-generated answer is ever produced without valid context.

The system prompt is a constraint, not a style guide. It prohibits answering from anything other than the retrieved context and bounds scope to machine health, anomaly explanation and recent alerts, excluding maintenance procedure and safety guidance. An optional cloud model may substitute for the local one when connectivity exists, reducing latency, and is held to the identical prompt constraint and the identical evaluation so that the offline path is never the unevaluated one.

VII. Evaluation Methodology

Table V states every metric and target before measurement. We set them out in full because a stated target that is later missed is informative, while a target chosen after the fact is not.

A. Partitioning and Baselines

Two methodological commitments govern the detection results. First, the anomaly threshold must be fitted on a partition disjoint from the one on which F1 is measured. Fitting a threshold on the same normal-class windows against which performance is then reported is a self-evaluation, and we do not report numbers obtained that way as held-out. Second, the remaining-useful-life split is by file rather than by window, for the leakage reason given in Section V.

Comparison is organised against four reference classes rather than a single baseline, because no prior system occupies the same design point. These are commercial twin platforms, as the incumbent alternative; edge and TinyML monitoring systems [16], [17], [20], as the resource-envelope reference; high-accuracy server-class bearing diagnosis [11], [13], [14], as the detection-accuracy ceiling; and cloud-hosted industrial language interfaces [6], [24]–[26], as the operator-interface reference. For each we report the figure the original authors published alongside our measured value, and we do not re-run their systems.

TABLE V
Evaluation Metrics and Pre-Registered Targets

Subsystem	Metric	Target
Hardware	Retrofit with no host-machine modification	pass or fail
Hardware	Vibration RMS against reference instrument	within $\pm 10\%$
Hardware	Temperature against traceable thermometer	within $\pm 1^{\circ}$C
Hardware	Total per-deployment cost	$\leq$ INR 20,000
Detection	Isolation Forest F1, held-out CWRU split	≥ 0.80
Detection	Autoencoder F1, held-out CWRU split	≥ 0.85
Detection	OOD flag recall on synthetic cases	≥ 0.90
Prognostics	RUL error on file-level held-out IMS split	reported, not gated
Runtime	Combined model size	≤ 5 MB
Runtime	Inference latency per reading	≤ 100 ms
Runtime	Dashboard load over local network	≤ 3 s
Runtime	Copilot latency, local SLM	≤ 15 s
Copilot	Response consistency with retrieved state, 50 queries	$\geq 90\%$
Copilot	Fallback trigger rate, insufficient context	100%
System	Full function with no network egress	pass or fail
System	ROI projection against manual calculation	within $\pm 5\%$
Research	Retrofit reproduced by a second installer	pass or fail

VIII. Results

This section is structured for the evaluation campaign, and cells that depend on the pilot machine or on the held-out detection partitions remain marked TBD. Three groups of result are reported now, because none of them depends on either. The exported artefact sizes are properties of the trained models and are measured directly from the deployed files. The prognostic figures were computed on a file-level held-out IMS partition from the outset, so they are unaffected by the self-evaluation issue described next, although they remain a public-

TABLE VI
DETECTION AND PROGNOSTIC PERFORMANCE

Model	Metric	Target	Measured
Isolation Forest	F1	≥ 0.80	TBD
Isolation Forest	Precision	not set	TBD
Isolation Forest	Recall	not set	TBD
1D autoencoder	F1	≥ 0.85	TBD
1D autoencoder	Precision	not set	TBD
1D autoencoder	Recall	not set	TBD
Fused health index	AUROC	not set	TBD
OOD gate	Recall	≥ 0.90	TBD
OOD gate	False-flag rate	not set	TBD
Ridge RUL	MAE	reported	0.247
Ridge RUL	R^2	reported	0.187

Prognostic figures are computed on the file-level held-out IMS partition over 320 windows from 31 snapshots, with predictions clamped to $[0, 1]$ as the runtime does. See Fig. 4.

dataset result rather than a field result. The device-identity check was exercised by the first physical wiring attempt.

Preliminary *detection* figures obtained during pipeline development exist in the project record but are withheld here, because they were produced with the threshold fitted on the same partition used for scoring, which does not satisfy the held-out requirement stated in Section VII. Reporting them as held-out results would misrepresent them. Architectural and cost properties, which are determined by the design rather than by measurement, are reported as established.

A. Detection and Prognostic Performance

Table VI reports detection performance on the held-out CWRU partition and prognostic performance on the file-level held-out IMS partition. Fig. 4 shows the prognostic result in full, over 320 held-out windows drawn from 31 snapshot files.

The calibration panel makes the character of the result visible in a way the summary statistics do not. Predictions track the true life fraction only weakly across the mid-life range and compress toward the centre of the interval, and the residual panel shows the consequence as a monotonic band running from systematic over-prediction near failure to systematic under-prediction at full life. This is regression toward the mean, and it is the honest shape of an R^2 of 0.187. We report the channel as a life-fraction proxy carrying real but modest signal, and Table V accordingly records it as reported rather than gated.

B. Edge Runtime and Resource Cost

Table VII reports on-device cost, measured on the deployment hardware rather than estimated from the development machine. The artefact sizes are settled, and the combined footprint clears the 5 MB budget by a factor of roughly five and a half, which is what justifies the claim in Section V that the prognostic channel is effectively free.

C. Copilot Grounding and Refusal Behaviour

Table VIII reports copilot behaviour across the three context regimes defined by the validity gate, for both inference modes. The nominal and elevated regimes test whether the answer is consistent with the retrieved state, and the insufficient regime tests whether the deterministic fallback always fires.

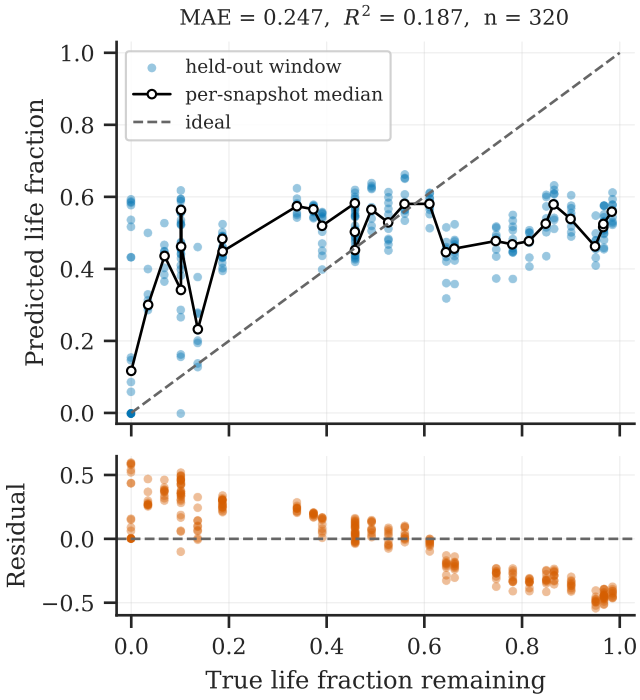


Fig. 4. Remaining-useful-life calibration on the file-level held-out IMS partition. Upper panel: predicted against true life fraction for each of 320 held-out windows, with the per-snapshot median overlaid and the diagonal marking ideal agreement. Lower panel: residuals against true life fraction. The monotonic residual band is regression toward the mean, with the model over-predicting remaining life near failure and under-predicting it at full life.

TABLE VII
EDGE RUNTIME MEASUREMENTS ON RASPBERRY PI 4

Quantity	Target	Measured
Isolation Forest artefact size	not set	823 KB
Autoencoder artefact size	not set	57 KB
RUL artefact size	not set	252 B
Combined model size	≤ 5 MB	0.86 MB
Inference latency per reading	≤ 100 ms	TBD
Feature extraction latency per burst	not set	TBD
Achieved acquisition sample rate	not set	800 Hz
Steady-state resident memory	not set	TBD
Peak memory with copilot loaded	< 4 GB	TBD
Copilot cold-load time	not set	TBD
Dashboard load time	≤ 3 s	TBD
Continuous uptime, soak test	not set	TBD

Artefact sizes are the deployed ONNX files, with the autoencoder figure covering its graph and external weight file together. The combined figure includes the calibration manifest.

D. Hardware Retrofit Validation

Table IX reports sensor accuracy against reference instruments and the reproducibility of the retrofit protocol. One row is settled. The device-identity read failed on the first wiring attempt, and the cause was the unsoldered header described in Section IV. We report the failure rather than the eventual pass, because the check earning its place in the protocol is the

TABLE VIII
COPILOT RESPONSE CONSISTENCY AND REFUSAL BEHAVIOUR

Measure	Target	Local	Cloud
Nominal context			
Response consistency	$\geq 90\%$	TBD	TBD
Mean latency	≤ 15 s	TBD	TBD
Elevated-anomaly context			
Response consistency	$\geq 90\%$	TBD	TBD
Correct field identification	not set	TBD	TBD
Insufficient context			
Fallback trigger rate	100%	TBD	TBD
Spurious generated answers	0	TBD	TBD
Aggregate			
Consistency over 50 queries	$\geq 90\%$	TBD	TBD

TABLE IX
RETROFIT AND SENSOR VALIDATION

Check	Target	Result
Device identity read on first wiring	pass	fail
Vibration RMS against reference	$\pm 10\%$	TBD
Temperature against thermometer	$\pm 1°$C	TBD
Installation without host modification	pass	TBD
Baseline recording duration achieved	30 min	TBD
Second-installer reproduction	pass	TBD
Full function with network disconnected	pass	TBD

The identity read failed on first wiring because of an unsoldered breakout header, and the retrofit protocol of Section IV was amended as a result. Remaining rows await the pilot-machine campaign.

finding: a module whose bus connection is intermittent returns a well-formed all-zero identity, and without an explicit identity gate that condition propagates as plausible data rather than as an error.

E. Comparison with Existing Approaches

Table X places the system against the four reference classes of Section VII. Published figures are attributed to their sources, and our measured column awaits the evaluation campaign. The comparison separates two kinds of claim, and the separation is the point. Rows in the upper block are *architectural*, following from the design and already settled. Rows in the lower block are *performance*, empirical and not yet in hand.

On the architectural rows the advantage is structural rather than incremental. Against commercial twin platforms, the system requires no industrial protocol on the asset, which is what makes a non-instrumented lathe addressable at all. It carries no licence or consumption charge against a sector with the credit constraint documented in [37], and it functions with the network disconnected. Against the cloud-hosted industrial language interfaces of [6], [24]–[26], it is the only one of the group whose language interface runs on the device, which removes both the recurring query cost and the connectivity dependency. Against the TinyML monitoring systems of [16], [17], [20], it concedes resource efficiency, since a Raspberry Pi is a far larger budget than an ESP32 or a self-powered node, and spends that budget on what those systems do not attempt: an operator-facing interface and a prognostic channel integrated into a complete twin.

On the performance rows we expect to concede as well as gain, and say so in advance. Server-class bearing diagnosis reaches 99.83% on CWRU under noise-free conditions with an autoencoder denoising front-end [13], and parallel CNN and LSTM architectures report comparably high accuracy across datasets [14]. We do not expect to match those figures, since our detectors are an order of magnitude smaller, consume four statistical features rather than high-resolution spectra, and run against an 800 Hz acquisition path instead of a 12 kHz laboratory rig. The relevant question is not whether a Raspberry Pi matches a GPU, but whether the achieved accuracy clears the threshold at which early warning changes a maintenance decision, which is what the targets in Table V encode.

F. Cost of Ownership

Table XI compares five-year cost of ownership for a single monitored asset. The hardware figure is from Table IV, and the software and connectivity figures are structural zeros, since the stack is open source and the system has no required egress.

IX. DISCUSSION

Three aspects of the design seem to us to generalise beyond this deployment.

Narrow scope is an enabling constraint. Accepting a single asset does not merely reduce cost. It removes whole categories of machinery. There is no asset registry, no cross-asset correlation, no multi-tenancy and no ontology to curate, and the maintenance burden of such an ontology is precisely what makes knowledge-graph-driven twins impractical for a firm with no IT staff. The state model collapses to one object with a single writer, which in turn is what makes the copilot's retrieval context small enough to fit entirely in a sub-2B-parameter model's prompt without any embedding infrastructure. The chain from P1 to P3 is causal, not coincidental.

Refusal is a feature of the interface, not a failure of it. The validity gate is the component we would defend most strongly. An interface that speaks plain language to a user who cannot cross-check it against a chart has a different error profile from a dashboard, since the failure mode is not confusion but misplaced confidence. Routing invalid context to a deterministic refusal, and flagging rather than suppressing uncertain predictions, makes the system's ignorance visible at exactly the moments when a fluent guess would be most harmful. The literature's caution about LLM reliability in industrial loops [27] points the same way, and we read it as an argument for architectural constraint rather than for better prompting.

Cost structure, not unit cost, is what excludes MSMEs. The adoption literature consistently identifies cost as a dominant barrier [33], [34], but the figure that matters is not the sticker price. It is the conversion of a capital purchase into an operating subscription. A firm that can find INR 20,000 once may still be unable to commit to a recurring per-query or per-asset charge of unknown escalation. Designing for zero recurring cost is therefore a different objective from designing for low cost, and it is the one that the edge-first property is in service of.

TABLE X
COMPARISON WITH REPRESENTATIVE ALTERNATIVES ACROSS THE FOUR REFERENCE CLASSES

Dimension	**Commercial twin platforms**	**Edge and TinyML monitors [16], [17], [20]**	**Server-class diagnosis [11], [13], [14]**	**Cloud industrial LLM [6], [24]–[26]**	**Digital Cousin (this work)**
Architectural properties, determined by design					
Asset data interface required	OPC-UA, MTConnect or vendor stack	raw sensor	dataset or raw sensor	MTConnect or IIoT database	raw sensor, retrofitted
Deployment scope	plant or fleet	single asset	single asset	machine or cell	single asset
Inference location	cloud or on-premise server	microcontroller	server or GPU	cloud API	edge node, on-device
Language interface	none, dashboard only	none	none	cloud-hosted	on-device SLM
Operates with no connectivity	no	yes	not applicable	no	yes
Recurring cost	subscription or consumption	none	none	per-query	none
One-time hardware cost	not applicable	lower than this work	server-class	not applicable	INR 20,020
Prognostic channel	yes	[20] only	varies	no	yes, life fraction
Confidence and OOD gating	varies	no	no	no	yes, two-tailed
Performance, published figures against pending measurement					
Detection accuracy	not published per deployment	[16]: unsupervised, no F1 given	[13]: 99.83% on CWRU	not applicable	TBD
Inference latency	network-bound	[16]: under 16 ms; [20]: under 1 s	millisecond class on GPU	[25]: API-bound	TBD
Memory footprint	server-class	[16]: 80 KB; [20]: under 5 MB	gigabyte class	cloud-hosted	0.86 MB of models
Model parameters	not published	[20]: 0.15 M	10^6 to 10^8	10^9 to 10^{11}	TBD
Query or task accuracy	not applicable	not applicable	not applicable	[25]: 93.3%; [24]: 49.4% gain	TBD
Refusal rate on invalid context	not applicable	not applicable	not applicable	not reported	TBD

Published figures are those reported by the cited authors under their own experimental conditions and are not re-measured here. They are not directly commensurable with ours, since sampling rates, hardware and partitioning differ. Commercial platform characteristics reflect the deployment model these products document rather than any single price quotation, which is itself part of the accessibility problem, since no uniform per-deployment price is publicly available.

TABLE XI
FIVE-YEAR COST OF OWNERSHIP, ONE MONITORED ASSET

Cost component	**This work**	**Cloud alternative**
Hardware, one-time	INR 20,020	varies
Software licence, annual	INR 0	subscription
Cloud charges, annual	INR 0	consumption
Connectivity, annual	INR 0	required
Integration labour	retrofit protocol	varies
Five-year total	**INR 20,020**	varies

Excludes electricity and the operator time to perform the retrofit. The comparison column is left unquantified deliberately, since commercial terms are quote-based and vary by fleet size.

X. LIMITATIONS

We list these as binding constraints on how the results should be read, not as caveats to be discounted.

Sampling bandwidth. The accelerometer's ceiling is 3,200 Hz and the achieved acquisition rate is approximately 800 Hz in 64-sample bursts. High-frequency bearing signatures above roughly 5 kHz, and the envelope analysis that targets them [9], are therefore unavailable. The system detects low-frequency and mid-frequency mechanical degradation through amplitude and shape statistics and should not be represented as performing spectral bearing diagnosis.

Dataset-to-deployment distribution shift. CWRU was collected at 12 kHz and IMS at 20 kHz on research-grade rigs. Training windows are matched to deployment windows by sample count rather than physical duration, so the two span different timescales. We argue the four features used are amplitude and shape statistics rather than timescale-dependent quantities, but this is an argument, not a validated equivalence, and the pilot-machine baseline recording is what will test it.

Prognostic accuracy is modest and dataset-bound. The remaining-useful-life channel reports an R^2 of 0.187 on the held-out IMS partition. Fig. 4 shows the model compressing its predictions toward the centre of the interval, so the channel is a weak ranking signal over a degradation trajectory rather than a dependable estimate at any single point. It is also a public-dataset result: the target is a position in a chronological sequence, not a measured time to failure, and nothing here establishes that the relationship transfers to the pilot machine.

Numeric faithfulness is not formally measured. The copilot is evaluated for response consistency against retrieved state and for refusal behaviour, as Table V records. A formal protocol for verifying that every numeric claim in a generated answer traces back to a field of the retrieved telemetry is beyond the present scope and is set out as future work in Section XI.

Partial hardware bring-up at the time of writing. The temperature channel is verified end to end on the deployment hardware, with the probe enumerating on the 1-Wire bus

and live readings reaching the dashboard served from the Pi. Vibration acquisition was blocked for part of the bring-up by the unsoldered-header fault described in Section IV. Every figure in Section VIII that depends on field vibration therefore awaits the pilot-machine campaign. The prognostic figures reported there are computed on the public IMS partition and are unaffected by this, but they are a dataset result and no claim in this paper should be read as resting on measured field vibration.

Single asset, single site. The paradigm is defined around one machine, and the evaluation is a controlled pilot rather than a multi-site field study. Adoption friction, installer variability across real workshops, and the accuracy of the cost figures at scale are open questions that only field deployment with MSME partners will answer.

Copilot latency precludes control use. At 5 to 15 s per query the copilot is a diagnostic aid. It is read-only by design and must not be placed in any control loop.

Historical window bound. The history endpoint is bounded by a fixed row cap rather than a true time range, which at the default publish interval corresponds to well under a day. Extending it requires pagination and time-range support on the API.

XI. Conclusion and Future Work

We have presented Digital Cousin, a digital twin for a single legacy machine that runs entirely on a Raspberry Pi 4 and speaks to its operator in plain language. The design is organised around three properties, namely single-asset scope, edge-first execution and natural-language accessibility, which we argue constitute a coherent design point rather than a reduced enterprise twin, and which are reachable at approximately INR 20,000 of one-time hardware with no licence and no cloud dependency. The system retrofits a non-instrumented rotary asset with a MEMS accelerometer and a 1-Wire probe, derives anomaly score, calibrated health index, two-tailed confidence flag and a life-fraction prognostic on-device through ONNX artefacts governed by an explicit manifest, and exposes that state through a quantized SLM whose answers are constrained to retrieved telemetry and whose validity gate produces a deterministic refusal when context is missing, stale or out-of-distribution. Quantitative results against the methodology of Section VII are reported into the structure of Section VIII, where the model footprint and the prognostic channel are settled and the detection and field figures await the pilot-machine campaign.

Four directions follow, and the first is the one we consider most valuable. A dedicated faithfulness protocol for structured numeric telemetry would treat every numeric token in a generated answer as a claim to be traced back to a field of the retrieved state object. Existing faithfulness work [29]–[31] evaluates prose, where an unsupported claim is a proposition that can be checked by entailment. Telemetry breaks that assumption, because the atom of unfaithfulness is a wrong float, carrying no linguistic cue and sitting comfortably inside the range of plausible values. Building and validating such a protocol, including the entity linking needed to confirm that a retrieved number is attached to the correct quantity, is a substantial undertaking and is scoped as a separate phase of this work. Second, extending sensing with a non-invasive current clamp would add an electrically grounded fault channel, covering phase imbalance and overload, complementary to vibration and cheap to retrofit. Third, physics-informed constraints of the kind LiRUL applies to battery degradation [20] offer a route to a prognostic channel stronger than a four-feature linear model without leaving the edge budget, which Fig. 4 indicates is the channel most in need of strengthening. Fourth, a multi-site field study with MSME partners, mapping the bill of materials against the eligibility criteria of existing industrial subsidy schemes, is what would turn a working pilot into an adoption pathway, which the adoption literature identifies as the missing step rather than awareness [33]–[35].

Acknowledgment

The authors thank Dayananda Sagar University for institutional support. No external funding was received for this study.

References

[1] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, Apr. 2019.

[2] J. Lee, B. Bagheri, and H.-A. Kao, "A cyber-physical systems architecture for Industry 4.0-based manufacturing systems," *Manuf. Lett.*, vol. 3, pp. 18–23, Jan. 2015.

[3] R. van Dinter, B. Tekinerdogan, and C. Catal, "Reference architecture for digital twin-based predictive maintenance systems," *Comput. Ind. Eng.*, vol. 177, art. 109099, Mar. 2023.

[4] J. Leng, Q. Liu, S. Ye, J. Jing, Y. Wang, C. Zhang, D. Zhang, and X. Chen, "Digital twin-driven rapid reconfiguration of the automated manufacturing system via an open architecture model," *Robot. Comput.-Integr. Manuf.*, vol. 63, art. 101895, 2020.

[5] S. Krommes and F. Tomaschko, "Conceptual framework of a digital twin fostering sustainable manufacturing in a brownfield approach of small volume production for SMEs," in *Manufacturing Driving Circular Economy (GCSM 2022)*, Lecture Notes in Mechanical Engineering. Cham, Switzerland: Springer, 2023, pp. 519–527.

[6] A. Gautam, M. R. Aryal, S. Deshpande, S. Padalkar, M. Nikolaenko, M. Tang, and S. Anand, "IIoT-enabled digital twin for legacy and smart factory machines with LLM integration," *J. Manuf. Syst.*, vol. 80, pp. 511–523, 2025.

[7] W. A. Smith and R. B. Randall, "Rolling element bearing diagnostics using the Case Western Reserve University data: A benchmark study," *Mech. Syst. Signal Process.*, vols. 64–65, pp. 100–131, 2015.

[8] H. Qiu, J. Lee, J. Lin, and G. Yu, "Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics," *J. Sound Vib.*, vol. 289, no. 4–5, pp. 1066–1090, 2006.

[9] R. B. Randall and J. Antoni, "Rolling element bearing diagnostics: A tutorial," *Mech. Syst. Signal Process.*, vol. 25, no. 2, pp. 485–520, 2011.

[10] R. Zhao, R. Yan, Z. Chen, K. Mao, P. Wang, and R. X. Gao, "Deep learning and its applications to machine health monitoring," *Mech. Syst. Signal Process.*, vol. 115, pp. 213–237, 2019.

[11] H.-X. Hu, Y. Feng, Q. Hu, and Y. Zhang, "A masked one-dimensional convolutional autoencoder for bearing fault diagnosis based on digital twin enabled industrial internet of things," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3242–3253, 2023.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

[13] F. Yuan, C. Zhang, and X. Liu, "Bearing fault diagnosis based on auto-encoder combined with CNN," in *Fuzzy Systems and Data Mining IX*, Frontiers in Artificial Intelligence and Applications, vol. 354. Amsterdam, The Netherlands: IOS Press, 2023, pp. 334–344.

[14] G. Fu, Q. Wei, and Y. Yang, "Bearing fault diagnosis with parallel CNN and LSTM," *Math. Biosci. Eng.*, vol. 21, no. 2, pp. 2385–2406, 2024.

[15] E. Njor, M. A. Hasanpour, J. Madsen, and X. Fafoutis, "A holistic review of the TinyML stack for predictive maintenance," *IEEE Access*, vol. 12, pp. 184861–184882, 2024.

[16] M. Antonini, M. Pincheira, M. Vecchio, and F. Antonelli, "An adaptable and unsupervised TinyML anomaly detection system for extreme industrial environments," *Sensors*, vol. 23, no. 4, art. 2344, 2023.

[17] Z. Chen, Y. Gao, and J. Liang, "LOPdM: A low-power on-device predictive maintenance system based on self-powered sensing and TinyML," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–13, 2023.

[18] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, and S. Han, "Tiny machine learning: Progress and futures," *IEEE Circuits Syst. Mag.*, vol. 23, no. 3, pp. 8–34, 2023.

[19] S. Lu, J. Lu, K. An, X. Wang, and Q. He, "Edge computing on IoT for machine signal processing and fault diagnosis: A review," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11093–11116, 2023.

[20] J. Pournazari, M. El-Dalahmeh, A. Al-Dubai, X. Liu, D.-H. Park, J. Marco, T. Q. Dinh, J.-H. Ahn, and M. Faraji Niri, "LiRUL: A physics-constrained and lightweight learning framework for edge-based RUL prediction," *Results Eng.*, 2026.

[21] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLlama: An open-source small language model," arXiv:2401.02385, 2024.

[22] Qwen Team, "Qwen2.5 technical report," arXiv:2412.15115, 2024.

[23] S. V. Kandala, P. Medaranga, and A. Varshney, "TinyLLM: A framework for training and deploying language models at the edge computers," arXiv:2412.15304, 2024.

[24] T. An, Y. Zhou, H. Zou, and J. Yang, "IoT-LLM: A framework for enhancing large language model reasoning from real-world sensor data," *Patterns*, 2025.

[25] J. Jeon, Y. Sim, H. Lee, C. Han, D. Yun, and E. Kim, "ChatCNC: Conversational machine monitoring via large language model and real-time data retrieval augmented generation," *J. Manuf. Syst.*, vol. 79, pp. 504–514, 2025.

[26] H. Peng, J. Liu, J. Du, J. Gao, and W. Wang, "BearLLM: A prior knowledge-enhanced bearing health management framework with unified vibration signal representation," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 19, 2025, pp. 19866–19874.

[27] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for process simulation parametrization in digital twins," in *Proc. 29th IEEE Int. Conf. Emerg. Technol. Factory Autom. (ETFA)*, 2024; arXiv:2405.18092.

[28] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," arXiv:2505.02076, 2025.

[29] F. S. Bao, M. Li, R. Qu, G. Luo, E. Wan, Y. Tang, W. Fan, M. S. Tamber, S. Kazi, V. Sourabh, M. Qi, R. Tu, C. Xu, M. Gonzales, O. Mendelevitch, and A. Ahmad, "FaithBench: A diverse hallucination benchmark for summarization by modern LLMs," in *Proc. 2025 Conf. Nations Americas Chapter ACL (NAACL-HLT)*, vol. 2: Short Papers, 2025, pp. 448–461.

[30] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. Eur. Chapter ACL: System Demonstrations*, St. Julians, Malta, 2024, pp. 150–158.

[31] C. Niu, Y. Wu, J. Zhu, S. Xu, K. Shum, R. Zhong, J. Song, and T. Zhang, "RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models," in *Proc. 62nd Annu. Meeting ACL (Vol. 1: Long Papers)*, Bangkok, Thailand, 2024, pp. 10862–10878.

[32] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Trans. Inf. Syst.*, vol. 43, no. 2, pp. 1–55, 2025.

[33] A. Raj, G. Dwivedi, A. Sharma, A. B. L. de Sousa Jabbour, and S. Rajak, "Barriers to the adoption of industry 4.0 technologies in the manufacturing sector: An inter-country comparative perspective," *Int. J. Prod. Econ.*, vol. 224, art. 107546, 2020.

[34] K. L. Gold, "Effects of Industry 4.0 on small and medium-scale enterprises: An analytical and bibliometric review," *SAGE Open*, vol. 15, no. 2, 2025.

[35] S. O. Ooko and S. M. Karume, "Application of tiny machine learning in predicative maintenance in industries," *J. Comput. Theories Appl.*, vol. 2, no. 1, pp. 131–150, 2024.

[36] Ministry of Micro, Small and Medium Enterprises, Government of India, *Annual Report 2023–24*. New Delhi, India: Government of India, 2024.

[37] International Finance Corporation and Intellecap, *Financing India's MSMEs: Estimation of Debt Requirement of MSMEs in India*. Washington, DC, USA: World Bank Group, 2018.

[38] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[39] Z. Lu, X. Li, D. Cai, R. Yi, F. Liu, X. Zhang, N. D. Lane, and M. Xu, "Small language models: Survey, measurements, and insights," arXiv:2409.15790, 2024.